

Processes and Mailboxes — A Readable Companion to the OTP Docs

Chapter 2 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why processes are the unit of everything

An Erlang process is a tiny isolated computer: its own heap, its own mailbox, and no way to touch anyone else’s memory. Everything else in OTP, supervision included, is built on that isolation.

Processes are cheap enough to use freely. A new process takes 327 words of memory, 233 of them for its initial heap and stack. A million idle processes fit in a few gigabytes, so “one process per connection, per request, per game session” is the normal design, not an extravagance.

The price of isolation is copying. Sending a message copies the term into the receiver’s heap, with two exceptions on the same node: large (refc) binaries and literals are shared, not copied. This is what lets a crashed process vanish without corrupting anyone: nobody else holds pointers into its memory.

Share nothing, but pay per send

The mental model is postal, not shared-memory. You hand over a copy; you never get a lock. That makes reasoning local and garbage collection per-process (no global pauses), but it also means sending a 10 MB map to 100 processes costs 1 GB of copying.

2. The building blocks

Primitive	What it does	Direction	Notes
<code>spawn/1,3</code>	Starts a process	—	Failure goes unnoticed unless you link or monitor
<code>link/1, spawn_link</code>	Ties two lifetimes together	Two-way	One link per pair; repeated <code>link</code> has no effect
<code>monitor(process, Pid)</code>	Watches a process	One-way	Each call is a separate monitor; each sends its own <code>'DOWN'</code>
<code>register(Name, Pid)</code>	Gives a pid an atom name	—	Removed automatically when the process dies
<code>alias/0,1</code>	A reference that can receive messages	—	Lets a caller refuse late replies (used by <code>gen_server:call</code>)
<code>Pid ! Msg</code>	Sends a message	—	Never fails for a dead pid; the message is dropped

Links versus monitors

Use a link when two processes should live and die together; use a monitor when one only needs to know the other died. A monitor on a pid that is already dead delivers `{'DOWN', Ref, process, Pid, noproc}` at once, so there is no race between checking and watching.

The only ordering guarantee

If process A sends S1 and then S2 to B, S1 will not arrive after S2. That is all. There is no ordering across different senders and no causal ordering through intermediaries:

```
A — m1 —————> B
A — m2 —> C — m2 (forward) —> B    B may see m2 before m1
```

Receive

`receive` walks the mailbox from the oldest message, trying the clauses top to bottom against each message. The first message that matches any clause is removed; every other message stays where it was. `after T` fires if nothing matched within T milliseconds; `after 0` checks only what is already there.

3. How the runtime actually does it

The mailbox is a queue you scan

Each `receive` is $O(N)$ in the number of messages ahead of the one it matches. Unmatched messages are not errors; they just sit there and are rescanned on every later `receive`. In a busy process, a message nobody matches turns every `receive` into a longer walk, forever.

The reference trick

There is one pattern the compiler and runtime make cheap. If you create a reference and every clause of the next `receive` matches on it, the runtime remembers the queue position at creation and scans only messages that arrived after it:

```
call(Pid, Req) ->
  Ref = monitor(process, Pid),
  Pid ! {request, self(), Ref, Req},
  receive
    {reply, Ref, Result} ->
      demonitor(Ref, [flush]),
      Result;
    {'DOWN', Ref, process, Pid, Reason} ->
      exit(Reason)
  after 5000 ->
    demonitor(Ref, [flush]),
    exit(timeout)
end.
```

Compile with `erlc +recv_opt_info` to see whether a given `receive` got the optimization. `gen_server:call` uses this pattern, plus an alias (since OTP 24) so a reply arriving after the timeout never lands in the caller's mailbox.

Where messages live

By default, messages sit on the receiving process's heap and are scanned by its garbage collector. A process that receives heavy traffic can set `process_flag(message_queue_data, off_heap)` so queued messages live outside the heap and stop inflating GC work.

Memory controls

Tool	What it does	Use when
<code>min_heap_size</code> (in <code>spawn_opt</code>) or <code>+h</code>	Larger starting heap, fewer early GCs	Few processes, measured benefit only
<code>erlang:hibernate/3</code> or <code>hibernate</code> return	Shrinks the process to minimum memory until the next message	Long-idle processes
<code>max_heap_size</code> flag	Kills and/or logs a process whose heap exceeds a limit	Guarding against runaway mailboxes or state

The docs warn that a larger initial heap also means fewer collections, which can keep large binaries alive longer than you want. Measure before tuning.

4. Plausible but wrong

4.1 A receive loop with no catch-all

```
loop(State) ->
  receive
    {add, X}    -> loop(State + X);
    {get, From} -> From ! State, loop(State)
  end.
```

Any other message (a stray reply, an `'EXIT'`, a typo like `{Add, 1}`) stays in the mailbox forever. Each loop iteration rescans it, so the process slows down and grows without ever failing. Fix: add a final clause that logs and drops unknown messages, or use `gen_server`, whose `handle_info` makes the catch-all explicit.

4.2 Waiting for a reply without a fresh reference

```
Pid ! {request, self(), Req},
receive
  {reply, Result} -> Result
end
```

With a busy mailbox this scans every queued message each time, which turns into quadratic behaviour under load. It can also pick up the wrong reply: one left over from an earlier request that timed out. Fix: tag each request with a new reference (or monitor) and match on it, as in the `call/2` example above.

4.3 Assuming ordering through a third process

```
%% in A
B ! {config, NewConfig},
C ! {apply_and_notify, B}.    %% C then sends B 'config_applied'
```

B may receive `config_applied` before `{config, ...}`. Only direct `A→B` order is guaranteed. Fix: route both messages through the same sender, or have B wait for the config message explicitly before acting on the notification.

4.4 Sending to a name that may not exist

```
notify(Event) ->
  logger_proc ! {event, Event}.
```

Sending to a pid never fails, but sending to an unregistered name raises `badarg`. While `logger_proc` is restarting, every caller of `notify/1` crashes. Fix: decide what should happen during the gap. Either let the caller crash knowingly, or use `whereis/1` and handle `undefined`, or go through a supervisor-owned API that absorbs restarts.

4.5 Retrying a timed-out call

```
charge(Card, Amount) ->
  try gen_server:call(payments, {charge, Card, Amount}, 2000)
  catch exit:{timeout, _} -> charge(Card, Amount)
  end.
```

Since OTP 24 the late reply is discarded safely, but the server may still have done the work. A timeout means “I stopped waiting”, not “it didn’t happen”. This code can charge twice. Fix: make the operation idempotent (a request id the server deduplicates), or treat the timeout as an unknown outcome and check state before retrying. Note also that a `gen_server` calling itself exits with `calling_self` rather than deadlocking.

4.6 Fan-out of a large term

```
[Pid ! {snapshot, BigMap} || Pid <- Workers]
```

Each send copies `BigMap` into another heap. With 1,000 workers and a 5 MB map, that is 5 GB of copying and 1,000 extra GC loads. Fix: put shared read-mostly data in ETS (readers copy only the rows they need), or send a key and let workers look it up. Large binaries are the exception: those over 64 bytes are shared by reference on the same node.

4.7 Fire-and-forget spawn

```
handle_cast({resize, Img}, State) ->
    spawn(fun() -> resize_and_store(Img) end),
    {noreply, State}.
```

If `resize_and_store/1` crashes, nobody knows; the image is just missing. If the parent stops, the worker keeps running unsupervised. Fix: start it under a `simple_one_for_one` (or `Task`) supervisor, or `spawn_monitor` it and handle the `'DOWN'` result.

5. Review checklist and sources

- ☐ Every hand-written `receive` loop has a catch-all clause or is replaced by a behaviour
- ☐ Every request/reply is tagged with a fresh reference or monitor, and matched on it
- ☐ No code relies on ordering except between one sender and one receiver
- ☐ Sends to registered names handle the “not registered right now” case
- ☐ Timeouts are treated as unknown outcomes; retried operations are idempotent
- ☐ Large shared data is in ETS or passed as binaries, not copied per send
- ☐ Every spawned process is linked, monitored, or supervised
- ☐ High-traffic processes have `message_queue_data` considered; heap tuning is measured

Sources

- Processes, Efficiency Guide: https://www.erlang.org/doc/system/eff_guide_processes.html
- Processes, Reference Manual: https://www.erlang.org/doc/system/ref_man_processes.html
- Expressions (receive), Reference Manual: <https://www.erlang.org/doc/system/expressions.html>
- `gen_server` module reference: https://www.erlang.org/doc/apps/stdlib/gen_server.html
- `erlang` module reference: <https://www.erlang.org/doc/apps/erts/erlang.html>